# Prácticas Evaluadas 1 y 2

Recuperación de contraseña con Java Swing y patrón DAO

**Estudiante:** Jason Azofeifa Flores

**Fecha:** 16 de octubre de 2026

**Tecnología:** Java 21, Swing (JFrame), SHA-256

# Práctica 1: Ventana "Olvidó mi contraseña"

## 1. Objetivo

Implementar una ventana independiente (JFrame) para la recuperación de contraseña, integrar un método en la capa DAO y validar los campos del formulario.

## 2. Cumplimiento de requerimientos

| Requerimiento | Implementación | Estado |
|---|---|---|
| JFrame llamado FrmOlvido | Clase FrmOlvido extends JFrame | CUMPLE |
| Ventana centrada | setLocationRelativeTo(null) | CUMPLE |
| Resizable = false | setResizable(false) | CUMPLE |
| Cierre sin finalizar el sistema | DISPOSE_ON_CLOSE | CUMPLE |
| Componentes para recuperar la clave | Campos Usuario y Correo, botones Recuperar y Cancelar | CUMPLE |
| Método en interfaz UsuarioDAO | boolean recuperarContrasena(String usuario, String correo) | CUMPLE |
| Método implementado en la clase DAO | UsuarioDAOImpl (estructura, sin consultas a BD) | CUMPLE |
| Campos obligatorios y espacios en blanco | Se aplica trim() y se rechazan vacíos | CUMPLE |
| Mensajes informativos | Mensaje en rojo dentro de la ventana | CUMPLE |
| Formato de correo | Expresión regular ^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ | CUMPLE |
| No continuar con errores | validarCampos() retorna antes de llamar al DAO | CUMPLE |

## 3. Evidencia




*Figura 1. Ventana de login con el enlace*

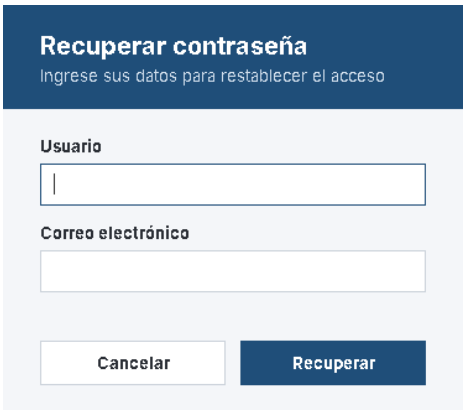



*Figura 2. FrmOlvido*




*Figura 3. Campos vacíos*




*Figura 4. Correo con formato inválido*

## 4. Pruebas ejecutadas

Las pruebas se ejecutaron de forma automática sobre las ventanas reales (pantalla virtual 1280x800). Resultado: 23/23 pruebas aprobadas.

| N.° | Prueba | Resultado |
|---|---|---|
| 1 | FrmLogin se muestra | APROBADA |
| 2 | Clase FrmOlvido es un JFrame | APROBADA |
| 3 | Ventana centrada en la pantalla | APROBADA |
| 4 | Resizable = false | APROBADA |
| 5 | Operacion de cierre = DISPOSE_ON_CLOSE | APROBADA |
| 6 | Campos vacios→ "El usuario es obligatorio y no puede quedar en blanco." | APROBADA |
| 7 | Usuario solo con espacios → "El usuario es obligatorio y no puede quedar en blanco." | APROBADA |
| 8 | Correo vacio → "El correo electrónico es obligatorio y no puede quedar en blanco." | APROBADA |
| 9 | Correo solo con espacios -> rechazado | APROBADA |
| 10 | Correo invalido "correo" -> rechazado | APROBADA |
| 11 | Correo invalido "correo@" -> rechazado | APROBADA |
| 12 | Correo invalido "@dominio.com" -> rechazado | APROBADA |
| 13 | Correo invalido "a@b" -> rechazado | APROBADA |
| 14 | Correo invalido "a b@c.com" -> rechazado | APROBADA |
| 15 | Correo invalido "a@b.." -> rechazado | APROBADA |
| 16 | Correo invalido "a@b.c" -> rechazado | APROBADA |
| 17 | Ventana sigue abierta mientras hay errores | APROBADA |
| 18 | Correo valido "a@b.co" -> procesa y cierra la ventana | APROBADA |
| 19 | Correo valido "nombre.apellido@dominio.com" -> procesa y cierra la ventana | APROBADA |

| N.° | Prueba | Resultado |
|---|---|---|
| 20 | Correo valido "x_y+z@sub.dominio.cr" -> procesa y cierra la ventana | APROBADA |
| 21 | DAO.recuperarContrasena(usuario, correo) devuelve true | APROBADA |
| 22 | Cerrar con X cierra solo FrmOlvido | APROBADA |
| 23 | El login principal sigue abierto tras cerrar FrmOlvido | APROBADA |

# Práctica 2: Regla de negocio de recuperación de contraseña

## 1. Objetivo

Declarar en la interfaz de autenticación el método de recuperación de contraseña e implementarlo en la capa DAO, almacenando la nueva contraseña con SHA-256 y un salt aleatorio, nunca en texto plano.

## 2. Diseño de la solución

- IAutenticacion: declara boolean recuperarContrasena(String correo, String nuevaContrasena).
- UsuarioDAO: interfaz que extiende IAutenticacion y agrega verificarContrasena.
- UsuarioDAOImpl: implementa la lógica y persiste en usuarios.txt con el formato correo;salt;hash.
- SeguridadClave: genera el salt (SecureRandom, 16 bytes) y calcula SHA-256(salt + contraseña).
- FrmOlvido: formulario con correo, nueva contraseña y confirmación, con validaciones.

Flujo: el usuario escribe su correo y la nueva contraseña; el formulario valida los campos; el DAO busca el correo, genera un salt nuevo, calcula el hash y reemplaza el registro. Si el correo no existe retorna false y la ventana lo informa. No se envían correos ni códigos de verificación.

## 3. Cumplimiento de requerimientos

| Requerimiento | Implementación | Estado |
|---|---|---|
| Método declarado en la interfaz | IAutenticacion / UsuarioDAO | CUMPLE |
| Implementación en la clase DAO | UsuarioDAOImpl.recuperarContrasena | CUMPLE |
| No almacenar en texto plano | Solo se guarda salt y hash | CUMPLE |
| SHA-256 con salt | MessageDigest SHA-256 sobre salt + contraseña | CUMPLE |
| Salt aleatorio | SecureRandom, 16 bytes, uno nuevo por cambio | CUMPLE |
| Salt almacenado junto al hash | Registro correo;salt;hash | CUMPLE |
| Recibe datos para identificar y registrar | Parámetros correo y nuevaContrasena | CUMPLE |
| Sin correos ni códigos | No implementado, fuera del alcance | CUMPLE |

## 4. Evidencia

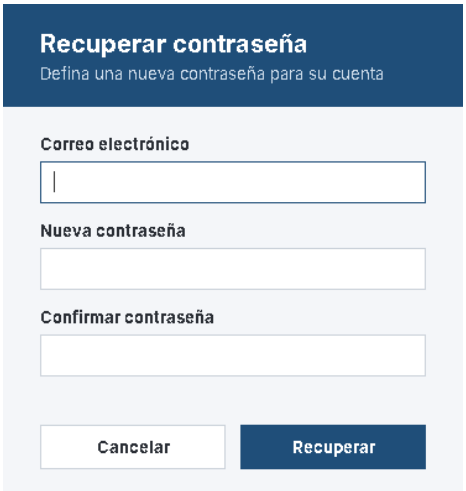



*Figura 5. FrmOlvido*




*Figura 6. Contraseñas distintas*




*Figura 7. Correo inválido*




*Figura 8. Correo no registrado*

Registro en usuarios.txt antes y después de recuperar la contraseña (se ven el salt y el hash, no la contraseña):

```
Registro antes:   demo@correo.com;iecrXGNnVdSjXSB8Z/jliw==;4L/uocFoAfXdHUapwUJFlDX9X7y4AJZlnALt25hYphA=
Registro despues: demo@correo.com;nWy3+obQbDHM+1a0pLoSqg==;V1A1fYT7FYtpYqncTlPIp1GiR/Asv/QhOOh2O3Ta4u8=
```

## 5. Pruebas ejecutadas

Resultado: 35/35 pruebas aprobadas. Incluye una verificación independiente que recalcula SHA-256(salt + contraseña) y comprueba que coincide con el hash guardado.

| N.° | Prueba | Resultado |
|---|---|---|
| 1 | FrmOlvido es JFrame centrado | APROBADA |
| 2 | Resizable = false | APROBADA |
| 3 | Cierre = DISPOSE_ON_CLOSE | APROBADA |
| 4 | UsuarioDAO extiende IAutenticacion | APROBADA |

| N.° | Prueba | Resultado |
|---|---|---|
| 5 | IAutenticacion declara recuperarContrasena(String,String) | APROBADA |
| 6 | UsuarioDAOImpl implementa UsuarioDAO | APROBADA |
| 7 | Todo vacio → "El correo electrónico es obligatorio." | APROBADA |
| 8 | Correo solo espacios -> rechazado | APROBADA |
| 9 | Correo con formato invalido → "El correo no tiene un formato válido (ejemplo: nombre@dominio.com)." | APROBADA |
| 10 | Contrasena vacia → "La nueva contraseña es obligatoria y no puede quedar en blanco." | APROBADA |
| 11 | Contrasena solo espacios -> rechazado | APROBADA |
| 12 | Contrasena menor a 8 caracteres → "La contraseña debe tener al menos 8 caracteres." | APROBADA |
| 13 | Sin confirmacion → "Debe confirmar la contraseña." | APROBADA |
| 14 | Confirmacion distinta → "Las contraseñas no coinciden." | APROBADA |
| 15 | Con errores el hash guardado no cambia (clave inicial Demo1234 sigue valida) | APROBADA |
| 16 | Ventana sigue abierta mientras hay errores | APROBADA |
| 17 | Correo inexistente→ "No existe una cuenta registrada con ese correo." | APROBADA |
| 18 | Flujo exitoso: cierra ventana y muestra confirmacion | APROBADA |
| 19 | El registro tiene formato correo;salt;hash | APROBADA |
| 20 | El archivo NO contiene la contrasena en texto plano | APROBADA |
| 21 | Salt nuevo distinto al anterior | APROBADA |
| 22 | Hash cambio | APROBADA |
| 23 | Salt de 16 bytes (Base64) | APROBADA |
| 24 | Hash SHA-256 de 32 bytes | APROBADA |
| 25 | Clave nueva inicia sesion | APROBADA |
| 26 | Clave anterior ya no sirve | APROBADA |
| 27 | Clave incorrecta rechazada | APROBADA |
| 28 | Hash == SHA-256(salt + contrasena) recalculado | APROBADA |
| 29 | Misma contrasena dos veces -> hash distinto (por salt aleatorio) | APROBADA |
| 30 | 1000 salts generados, todos distintos | APROBADA |
| 31 | DAO: correo null -> false | APROBADA |
| 32 | DAO: contrasena null -> false | APROBADA |
| 33 | DAO: correo inexistente -> false | APROBADA |
| 34 | DAO: contrasena con tildes y simbolos | APROBADA |

| N.° | Prueba | Resultado |
|---|---|---|
| 35 | Cerrar con X no termina el sistema (login sigue abierto) | APROBADA |

# Anexo: Código fuente

## Práctica 1

### UsuarioDAO.java

```
/**
 * Interfaz DAO para las operaciones relacionadas con usuarios.
 */
public interface UsuarioDAO {

    // ... aquí van los métodos que ya tenga su interfaz (login, registrar, etc.)

    /**
     * Declaración del método para recuperar la contraseña.
     *
     * @param usuario nombre de usuario ingresado en el formulario
     * @param correo  correo electrónico ingresado en el formulario
     * @return true si la solicitud de recuperación fue procesada
     */
    boolean recuperarContrasena(String usuario, String correo);
}
```

### UsuarioDAOImpl.java

```
/**
 * Implementación de UsuarioDAO.
 * Si su proyecto ya tiene una clase que implementa UsuarioDAO, agregue allí
 * únicamente el método recuperarContrasena.
 */
public class UsuarioDAOImpl implements UsuarioDAO {

    // ... aquí van los demás métodos ya existentes de la clase

    @Override
    public boolean recuperarContrasena(String usuario, String correo) {
        // Solo estructura e integración: la consulta a la base de datos
        // y el envío de correo quedan fuera del alcance de la práctica.
        return true;
    }
}
```

### FrmOlvido.java

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.util.regex.Pattern;

/**
 * Ventana "Olvidó mi contraseña".
 */
public class FrmOlvido extends JFrame {

    private static final Pattern EMAIL_PATTERN =
            Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");

    private final JTextField txtUsuario = new JTextField();
    private final JTextField txtCorreo = new JTextField();
    private final JLabel lblMensaje = new JLabel(" ");
    private final JButton btnRecuperar = Estilo.boton("Recuperar", true);
    private final JButton btnCancelar = Estilo.boton("Cancelar", false);

    private final UsuarioDAO usuarioDAO = new UsuarioDAOImpl();

    public FrmOlvido() {
        initComponents();
    }
```

```
private void initComponents() {
    setTitle("Olvidó mi contraseña");
    setDefaultCloseOperation(DISPOSE_ON_CLOSE); // no termina el sistema principal
    setResizable(false);                        // Resizable = false

    Estilo.campo(txtUsuario);
    Estilo.campo(txtCorreo);
    lblMensaje.setFont(Estilo.NORMAL);
    lblMensaje.setForeground(Estilo.ERROR);

    JPanel form = new JPanel(new GridBagLayout());
    form.setBackground(Estilo.FONDO);
    form.setBorder(new EmptyBorder(22, 30, 20, 30));
    GridBagConstraints g = new GridBagConstraints();
    g.gridx = 0;
    g.fill = GridBagConstraints.HORIZONTAL;
    g.anchor = GridBagConstraints.WEST;
    g.weightx = 1;

    g.gridy = 0; g.insets = new Insets(0, 0, 5, 0);
    form.add(Estilo.etiqueta("Usuario"), g);
    g.gridy = 1; g.insets = new Insets(0, 0, 14, 0);
    form.add(txtUsuario, g);
    g.gridy = 2; g.insets = new Insets(0, 0, 5, 0);
    form.add(Estilo.etiqueta("Correo electrónico"), g);
    g.gridy = 3; g.insets = new Insets(0, 0, 8, 0);
    form.add(txtCorreo, g);
    g.gridy = 4; g.insets = new Insets(0, 0, 14, 0);
    form.add(lblMensaje, g);

    JPanel botones = new JPanel(new GridLayout(1, 2, 12, 0));
    botones.setOpaque(false);
    botones.add(btnCancelar);
    botones.add(btnRecuperar);
    g.gridy = 5; g.insets = new Insets(0, 0, 0, 0);
    form.add(botones, g);

    JPanel contenido = new JPanel(new BorderLayout());
    contenido.add(Estilo.encabezado("Recuperar contraseña",
            "Ingrese sus datos para restablecer el acceso"), BorderLayout.NORTH);
    contenido.add(form, BorderLayout.CENTER);
    setContentPane(contenido);

    btnRecuperar.addActionListener(e -> recuperar());
    btnCancelar.addActionListener(e -> dispose());
    getRootPane().setDefaultButton(btnRecuperar);

    pack();
    setLocationRelativeTo(null); // centrada en la pantalla
}

private void recuperar() {
    String usuario = txtUsuario.getText().trim();
    String correo = txtCorreo.getText().trim();

    if (!validarCampos(usuario, correo)) {
        return; // impide continuar mientras existan errores
    }

    boolean ok = usuarioDAO.recuperarContrasena(usuario, correo);
    if (ok) {
        JOptionPane.showMessageDialog(this,
                "Su solicitud de recuperación fue procesada.",
                "Información", JOptionPane.INFORMATION_MESSAGE);
        dispose();
    } else {
        mostrarError("No se pudo procesar la solicitud.", null);
    }
}

/** Valida solo los campos del formulario (no consulta la base de datos). */
```

```
    private boolean validarCampos(String usuario, String correo) {
        txtUsuario.setBorder(Estilo.bordeCampo(Estilo.BORDE));
        txtCorreo.setBorder(Estilo.bordeCampo(Estilo.BORDE));

        if (usuario.isEmpty()) {
            mostrarError("El usuario es obligatorio y no puede quedar en blanco.", txtUsuario);
            return false;
        }
        if (correo.isEmpty()) {
            mostrarError("El correo electrónico es obligatorio y no puede quedar en blanco.",
txtCorreo);
            return false;
        }
        if (!EMAIL_PATTERN.matcher(correo).matches()) {
            mostrarError("El correo no tiene un formato válido (ejemplo: nombre@dominio.com).",
txtCorreo);
            return false;
        }
        lblMensaje.setText(" ");
        return true;
    }

    private void mostrarError(String mensaje, JTextField campo) {
        lblMensaje.setText("<html><body style='width:250px'>" + mensaje + "</body></html>");
        if (campo != null) {
            campo.setBorder(Estilo.bordeCampo(Estilo.ERROR));
            campo.requestFocusInWindow();
        }
        pack();
    }
}
```

# Práctica 2

## IAutenticacion.java

```
/**
 * Interfaz de autenticación: declara el método encargado de la
 * recuperación de contraseña.
 */
public interface IAutenticacion {

    /**
     * Registra una nueva contraseña para el usuario identificado por su correo.
     * La contraseña se guarda con SHA-256 + salt aleatorio, nunca en texto plano.
     *
     * @param correo          correo que identifica al usuario
     * @param nuevaContrasena nueva contraseña en texto plano (solo en memoria)
     * @return true si se actualizó; false si no existe un usuario con ese correo
     */
    boolean recuperarContrasena(String correo, String nuevaContrasena);
}
```

## UsuarioDAO.java

```
/**
 * Interfaz DAO de usuarios. Hereda recuperarContrasena de IAutenticacion.
 */
public interface UsuarioDAO extends IAutenticacion {

    @Override
    boolean recuperarContrasena(String correo, String nuevaContrasena);

    /** Comprueba una contraseña contra el hash y salt almacenados. */
    boolean verificarContrasena(String correo, String contrasena);
}
```

## SeguridadClave.java

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.util.Base64;

/** Utilidades de seguridad: salt aleatorio y hash SHA-256. */
public final class SeguridadClave {

    private static final int TAM_SALT = 16; // 128 bits
    private static final SecureRandom RANDOM = new SecureRandom();

    private SeguridadClave() { }

    /** Genera un salt aleatorio criptográficamente seguro (Base64). */
    public static String generarSalt() {
        byte[] salt = new byte[TAM_SALT];
        RANDOM.nextBytes(salt);
        return Base64.getEncoder().encodeToString(salt);
    }

    /** SHA-256(salt + contraseña), devuelto en Base64. */
    public static String hash(String contrasena, String saltBase64) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            md.update(Base64.getDecoder().decode(saltBase64));
            byte[] h = md.digest(contrasena.getBytes(StandardCharsets.UTF_8));
            return Base64.getEncoder().encodeToString(h);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 no disponible", e);
        }
    }
```

```
    /** Comparación en tiempo constante. */
    public static boolean coincide(String contrasena, String saltBase64, String hashEsperado) {
        byte[] a = hash(contrasena, saltBase64).getBytes(StandardCharsets.UTF_8);
        byte[] b = hashEsperado.getBytes(StandardCharsets.UTF_8);
        return MessageDigest.isEqual(a, b);
    }
}
```

### UsuarioDAOImpl.java

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.ArrayList;
import java.util.List;

/**
 * Implementación de UsuarioDAO. Persiste en un archivo de texto
 * (usuarios.txt) con el formato:  correo;salt;hash
 * La contraseña NUNCA se guarda en texto plano.
 */
public class UsuarioDAOImpl implements UsuarioDAO {

    private static final String SEP = ";";
    private final Path archivo;

    public UsuarioDAOImpl() {
        this(Paths.get("usuarios.txt"));
    }

    public UsuarioDAOImpl(Path archivo) {
        this.archivo = archivo;
        crearUsuarioDemoSiNoExiste();
    }

    @Override
    public synchronized boolean recuperarContrasena(String correo, String nuevaContrasena) {
        if (correo == null || nuevaContrasena == null) {
            return false;
        }
        String clave = correo.trim().toLowerCase();
        try {
            List<String> lineas = leer();
            boolean encontrado = false;
            for (int i = 0; i < lineas.size(); i++) {
                String[] p = lineas.get(i).split(SEP);
                if (p.length == 3 && p[0].equals(clave)) {
                    String salt = SeguridadClave.generarSalt(); // salt nuevo en cada cambio
                    String hash = SeguridadClave.hash(nuevaContrasena, salt);
                    lineas.set(i, clave + SEP + salt + SEP + hash);
                    encontrado = true;
                    break;
                }
            }
            if (encontrado) {
                Files.write(archivo, lineas, StandardCharsets.UTF_8);
            }
            return encontrado;
        } catch (IOException e) {
            return false;
        }
    }

    @Override
    public synchronized boolean verificarContrasena(String correo, String contrasena) {
        String clave = correo.trim().toLowerCase();
        try {
            for (String linea : leer()) {
                String[] p = linea.split(SEP);
                if (p.length == 3 && p[0].equals(clave)) {
```

```
                    return SeguridadClave.coincide(contrasena, p[1], p[2]);
                }
            }
        } catch (IOException ignorada) { }
        return false;
    }

    private List<String> leer() throws IOException {
        if (!Files.exists(archivo)) {
            return new ArrayList<>();
        }
        List<String> lineas = new ArrayList<>(Files.readAllLines(archivo,
StandardCharsets.UTF_8));
        lineas.removeIf(String::isBlank);
        return lineas;
    }

    /** Cuenta de prueba para poder demostrar el flujo (clave inicial: Demo1234). */
    private void crearUsuarioDemoSiNoExiste() {
        if (Files.exists(archivo)) {
            return;
        }
        String salt = SeguridadClave.generarSalt();
        String linea = "demo@correo.com" + SEP + salt + SEP + SeguridadClave.hash("Demo1234",
salt);
        try {
            Files.write(archivo, List.of(linea), StandardCharsets.UTF_8);
        } catch (IOException ignorada) { }
    }
}
```

### FrmOlvido.java

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.util.regex.Pattern;

/**
 * Ventana "Olvidó mi contraseña".
 */
public class FrmOlvido extends JFrame {

    private static final Pattern EMAIL_PATTERN =
            Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");

    private static final int MIN_CLAVE = 8;

    private final JTextField txtCorreo = new JTextField();
    private final JPasswordField txtNueva = new JPasswordField();
    private final JPasswordField txtConfirmar = new JPasswordField();
    private final JLabel lblMensaje = new JLabel(" ");
    private final JButton btnRecuperar = Estilo.boton("Recuperar", true);
    private final JButton btnCancelar = Estilo.boton("Cancelar", false);

    private final UsuarioDAO usuarioDAO = new UsuarioDAOImpl();

    public FrmOlvido() {
        initComponents();
    }

    private void initComponents() {
        setTitle("Olvidó mi contraseña");
        setDefaultCloseOperation(DISPOSE_ON_CLOSE); // no termina el sistema principal
        setResizable(false);                        // Resizable = false

        Estilo.campo(txtCorreo);
        Estilo.campo(txtNueva);
        Estilo.campo(txtConfirmar);
        lblMensaje.setFont(Estilo.NORMAL);
        lblMensaje.setForeground(Estilo.ERROR);
```

```
        JPanel form = new JPanel(new GridBagLayout());
        form.setBackground(Estilo.FONDO);
        form.setBorder(new EmptyBorder(22, 30, 20, 30));
        GridBagConstraints g = new GridBagConstraints();
        g.gridx = 0;
        g.fill = GridBagConstraints.HORIZONTAL;
        g.anchor = GridBagConstraints.WEST;
        g.weightx = 1;

        g.gridy = 0; g.insets = new Insets(0, 0, 5, 0);
        form.add(Estilo.etiqueta("Correo electrónico"), g);
        g.gridy = 1; g.insets = new Insets(0, 0, 14, 0);
        form.add(txtCorreo, g);
        g.gridy = 2; g.insets = new Insets(0, 0, 5, 0);
        form.add(Estilo.etiqueta("Nueva contraseña"), g);
        g.gridy = 3; g.insets = new Insets(0, 0, 14, 0);
        form.add(txtNueva, g);
        g.gridy = 4; g.insets = new Insets(0, 0, 5, 0);
        form.add(Estilo.etiqueta("Confirmar contraseña"), g);
        g.gridy = 5; g.insets = new Insets(0, 0, 8, 0);
        form.add(txtConfirmar, g);
        g.gridy = 6; g.insets = new Insets(0, 0, 14, 0);
        form.add(lblMensaje, g);

        JPanel botones = new JPanel(new GridLayout(1, 2, 12, 0));
        botones.setOpaque(false);
        botones.add(btnCancelar);
        botones.add(btnRecuperar);
        g.gridy = 7; g.insets = new Insets(0, 0, 0, 0);
        form.add(botones, g);

        JPanel contenido = new JPanel(new BorderLayout());
        contenido.add(Estilo.encabezado("Recuperar contraseña",
                "Defina una nueva contraseña para su cuenta"), BorderLayout.NORTH);
        contenido.add(form, BorderLayout.CENTER);
        setContentPane(contenido);

        btnRecuperar.addActionListener(e -> recuperar());
        btnCancelar.addActionListener(e -> dispose());
        getRootPane().setDefaultButton(btnRecuperar);

        pack();
        setLocationRelativeTo(null); // centrada en la pantalla
    }

    private void recuperar() {
        String correo = txtCorreo.getText().trim();
        String nueva = new String(txtNueva.getPassword());
        String confirmar = new String(txtConfirmar.getPassword());

        if (!validarCampos(correo, nueva, confirmar)) {
            return; // impide continuar mientras existan errores
        }

        // El DAO aplica SHA-256 + salt antes de guardar
        boolean ok = usuarioDAO.recuperarContrasena(correo, nueva);
        if (ok) {
            JOptionPane.showMessageDialog(this,
                    "Su contraseña fue actualizada correctamente.",
                    "Información", JOptionPane.INFORMATION_MESSAGE);
            dispose();
        } else {
            mostrarError("No existe una cuenta registrada con ese correo.", txtCorreo);
        }
    }

    /** Valida los campos del formulario. */
    private boolean validarCampos(String correo, String nueva, String confirmar) {
        for (JTextField c : new JTextField[]{txtCorreo, txtNueva, txtConfirmar}) {
            c.setBorder(Estilo.bordeCampo(Estilo.BORDE));
        }
```

```
        if (correo.isEmpty()) {
            mostrarError("El correo electrónico es obligatorio.", txtCorreo);
            return false;
        }
        if (!EMAIL_PATTERN.matcher(correo).matches()) {
            mostrarError("El correo no tiene un formato válido (ejemplo: nombre@dominio.com).",
txtCorreo);
            return false;
        }
        if (nueva.trim().isEmpty()) {
            mostrarError("La nueva contraseña es obligatoria y no puede quedar en blanco.",
txtNueva);
            return false;
        }
        if (nueva.length() < MIN_CLAVE) {
            mostrarError("La contraseña debe tener al menos " + MIN_CLAVE + " caracteres.",
txtNueva);
            return false;
        }
        if (confirmar.isEmpty()) {
            mostrarError("Debe confirmar la contraseña.", txtConfirmar);
            return false;
        }
        if (!nueva.equals(confirmar)) {
            mostrarError("Las contraseñas no coinciden.", txtConfirmar);
            return false;
        }
        lblMensaje.setText(" ");
        return true;
    }

    private void mostrarError(String mensaje, JTextField campo) {
        lblMensaje.setText("<html><body style='width:250px'>" + mensaje + "</body></html>");
        if (campo != null) {
            campo.setBorder(Estilo.bordeCampo(Estilo.ERROR));
            campo.requestFocusInWindow();
        }
        pack();
    }
}
```